

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет
Лабораторная работа 2
Реализация микросервисов

Выполнил:
Катков Алексей Сергеевич
Группа К3339

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Опираясь на документ, сформированный в рамках домашнего задания 4, необходимо реализовать разделение существующего backend-приложения на микросервисы. В качестве исходного приложения использовался сервис обмена рецептами, разработанный на Express.js, TypeORM и PostgreSQL.

В рамках лабораторной работы требовалось выполнить физическое разделение монолитного приложения на несколько сервисов, настроить отдельные базы данных для каждого сервиса, реализовать API Gateway и проверить работу полученной системы.

Ход работы

1. Исходное приложение

До выполнения лабораторной работы приложение представляло собой монолитный backend. В одной кодовой базе находились контроллеры, модели и бизнес-логика для пользователей, рецептов, ингредиентов, комментариев, лайков, сохраненных рецептов и подписок. Все сущности использовали одну общую базу данных PostgreSQL.

Исходное приложение включало следующие основные сущности:

- User;
- Follow;
- Recipe;
- Ingredient;
- DishType;
- RecipeStep;
- Comment;
- Like;
- SavedRecipe.

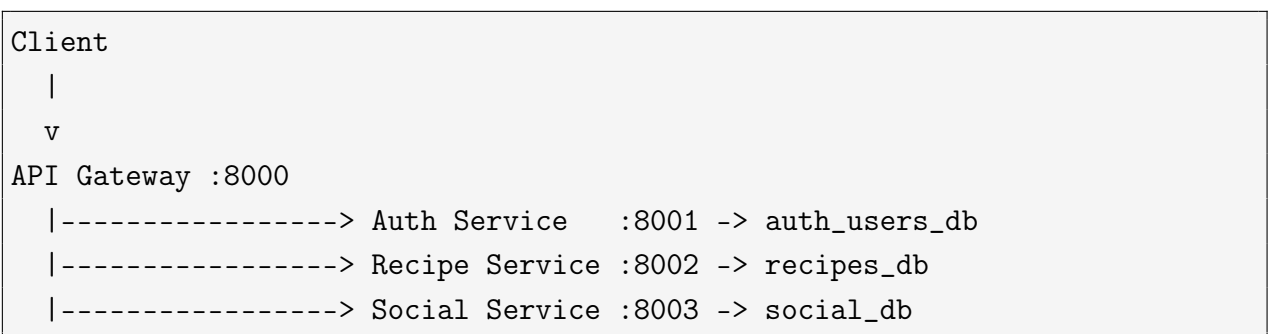
Главная проблема монолитного варианта заключалась в сильной связанности частей приложения. Например, рецепты напрямую ссылались на пользователей, а комментарии и лайки напрямую ссылались на рецепты и пользователей через связи TypeORM. Для микросервисной архитектуры такой подход не подходит, так как каждый сервис должен владеть своей областью данных.

2. Выделение микросервисов

Приложение было разделено на три основных микросервиса и API Gateway:

- Auth Service - регистрация, авторизация, пользователи и подписки;
- Recipe Service - рецепты, ингредиенты, типы блюд и шаги приготовления;
- Social Service - комментарии, лайки и сохраненные рецепты;
- API Gateway - единая точка входа для клиентских запросов.

Итоговая схема взаимодействия выглядит следующим образом:



API Gateway принимает клиентские запросы и перенаправляет их в нужный сервис. Такой подход скрывает внутреннюю структуру системы от клиента и позволяет обращаться к приложению через единую точку входа.

3. Распределение сущностей по сервисам

В результате разделения сущности были распределены следующим образом:

Сервис	Сущности
Auth Service	User, Follow

Recipe Service	Recipe, Ingredient, DishType, RecipeStep
Social Service	Comment, Like, SavedRecipe

Также были изменены связи между сущностями. Прямые связи TypeORM между разными сервисами были заменены на обычные идентификаторы. Например, в Recipe Service вместо связи с User хранится поле authorId, а в Social Service вместо связей с User и Recipe хранятся поля userId и recipeId.

Это соответствует подходу database-per-service: сервис не должен напрямую обращаться к таблицам другого сервиса.

4. Разделение баз данных

Для каждого сервиса была создана отдельная база данных PostgreSQL:

- auth_users_db - база данных Auth Service;
- recipes_db - база данных Recipe Service;
- social_db - база данных Social Service.

В базе Auth Service были созданы таблицы user и follow.

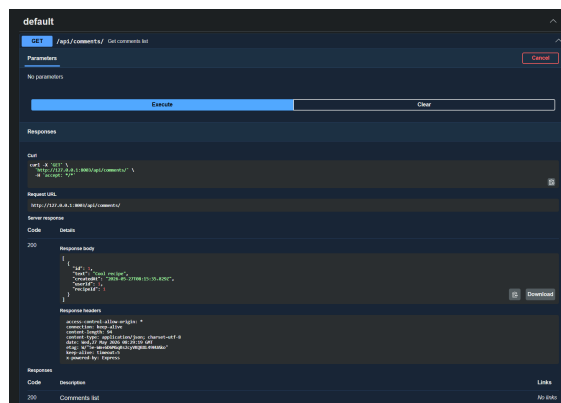


Рис. 1: Таблицы базы данных Auth Service

В базе Recipe Service были созданы таблицы recipe, ingredient, dish_type, recipe_step, а также промежуточные таблицы для связей рецептов с ингредиентами и типами блюд.

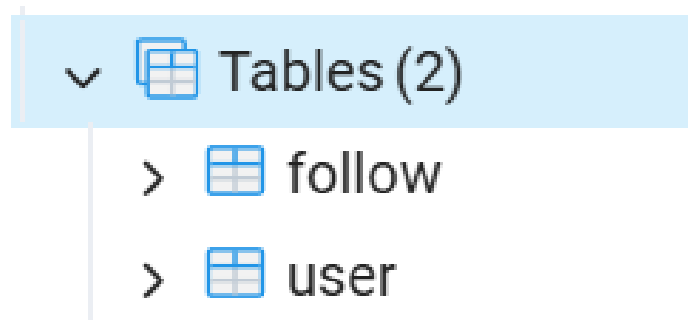


Рис. 2: Таблицы базы данных Recipe Service

В базе Social Service были созданы таблицы comment, like и saved_recipe.

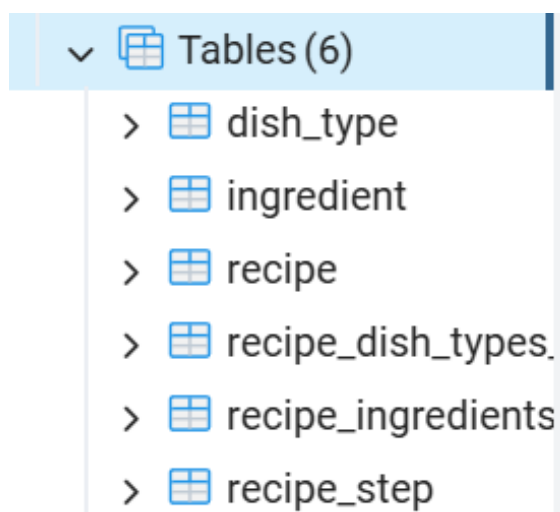


Рис. 3: Таблицы базы данных Social Service

5. Настройка API Gateway

API Gateway был реализован как отдельное Express-приложение. Он работает на порту 8000 и перенаправляет запросы к нужным микросервисам.

Маршрутизация была настроена следующим образом:

Путь	Сервис
/api/auth, /api/users, /api/follows	Auth Service
/api/recipes, /api/ingredients, /api/dish-types, /api/recipe-steps	Recipe Service

/api/comments, /api/likes, /api/saved- recipes	Social Service
--	----------------

Для проверки доступности gateway был реализован endpoint:

```
GET /health
```

Он возвращает ответ о состоянии API Gateway.

6. Документация API через Swagger

Для каждого сервиса была добавлена Swagger-документация. Документация доступна по адресам:

- <http://127.0.0.1:8001/docs> - Auth Service;
- <http://127.0.0.1:8002/docs> - Recipe Service;
- <http://127.0.0.1:8003/docs> - Social Service.

На рисунке ниже показана документация Auth Service.

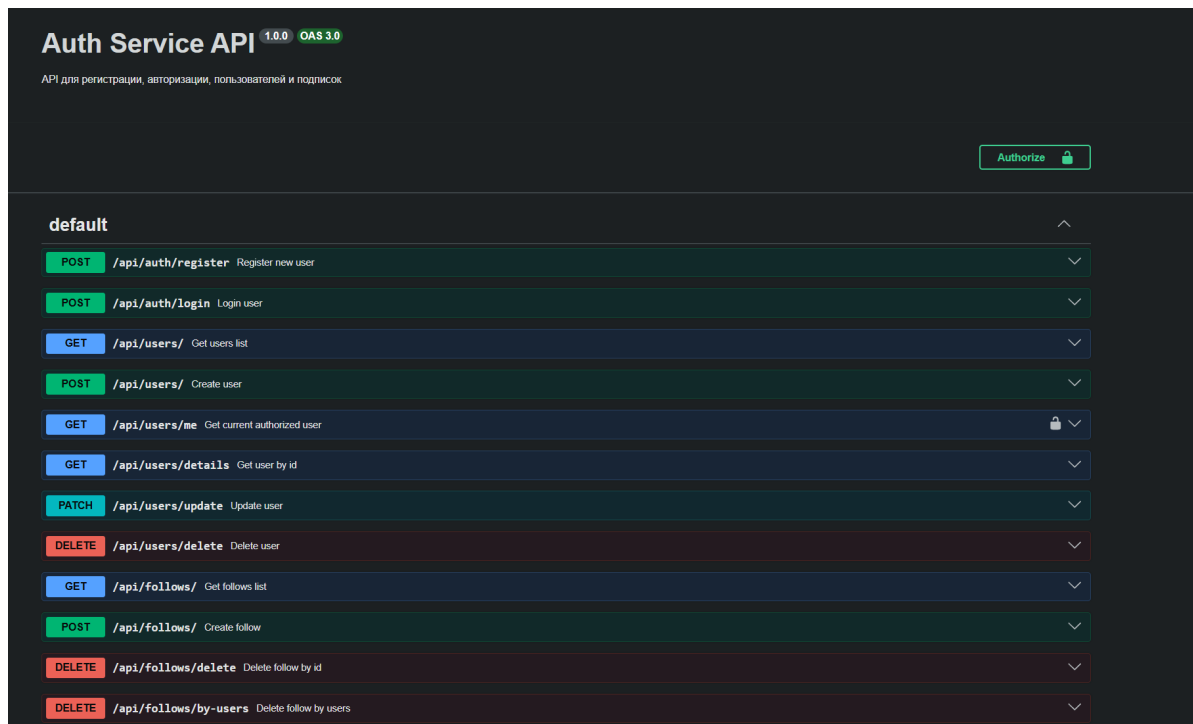


Рис. 4: Swagger-документация Auth Service

На следующем рисунке показана документация Recipe Service.

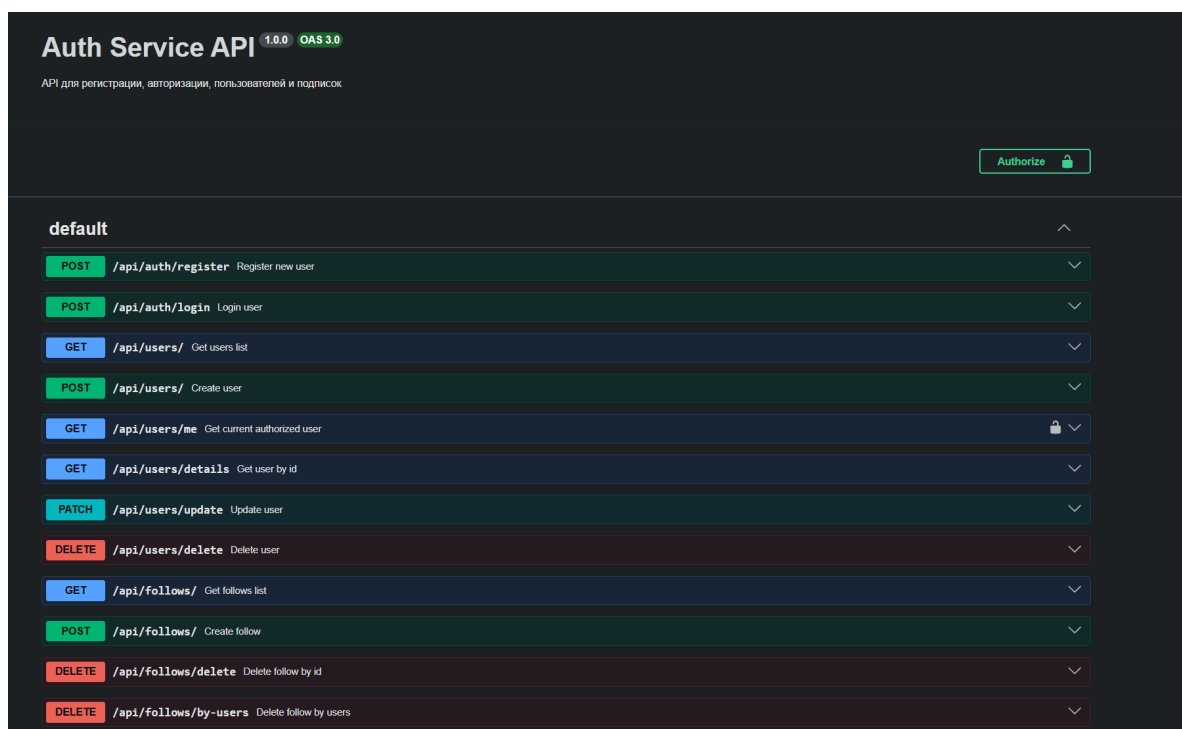


Рис. 5: Swagger-документация Recipe Service

Также была добавлена документация для Social Service.

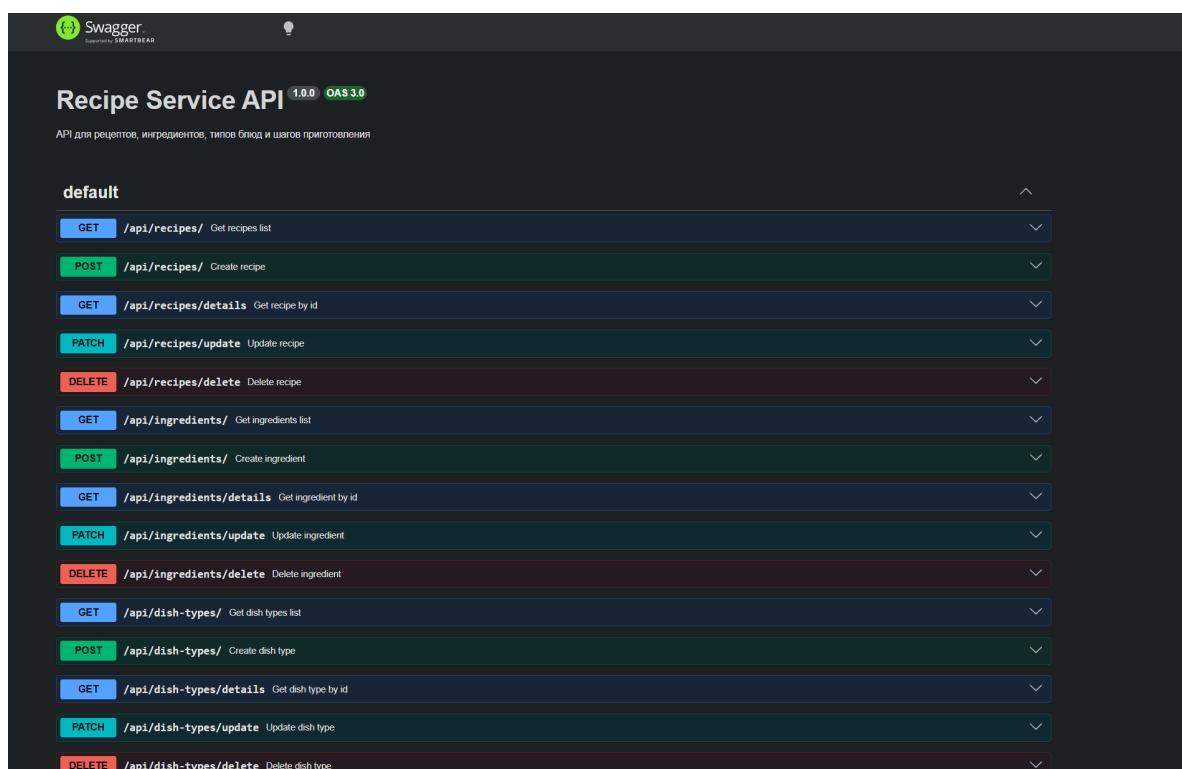


Рис. 6: Swagger-документация Social Service

7. Проверка работы микросервисов

Проверка выполнялась через Swagger UI и через PowerShell с использованием команды `Invoke-RestMethod`.

Сначала были проверены health-check endpoints всех сервисов:

```
Invoke-RestMethod http://127.0.0.1:8001/health
Invoke-RestMethod http://127.0.0.1:8002/health
Invoke-RestMethod http://127.0.0.1:8003/health
Invoke-RestMethod http://127.0.0.1:8000/health
```

После этого были проверены основные GET-запросы напрямую к сервисам и через API Gateway:

```
Invoke-RestMethod http://127.0.0.1:8000/api/users/
Invoke-RestMethod http://127.0.0.1:8000/api/recipes/
Invoke-RestMethod http://127.0.0.1:8000/api/comments/
```

В Auth Service был создан пользователь. После этого endpoint получения списка пользователей вернул созданную запись.

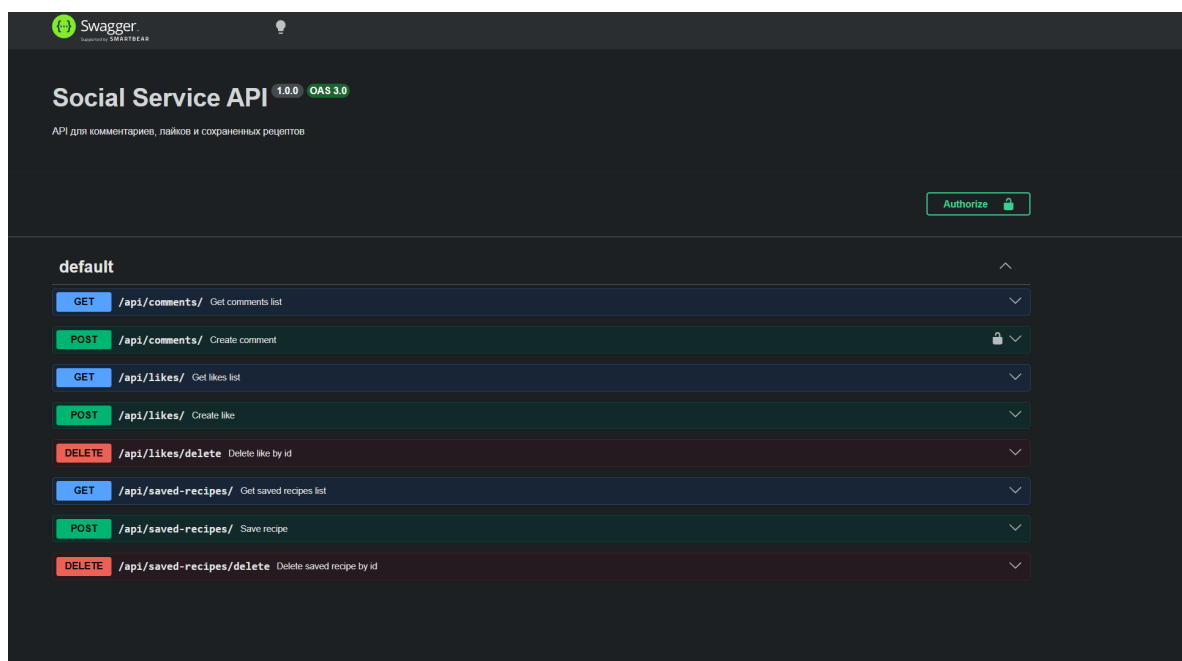


Рис. 7: Проверка получения пользователей в Auth Service

В Recipe Service были созданы ингредиент и тип блюда. Затем был создан рецепт с использованием созданных сущностей. Дополнительно через Swagger был выполнен запрос получения ингредиента по идентификатору.

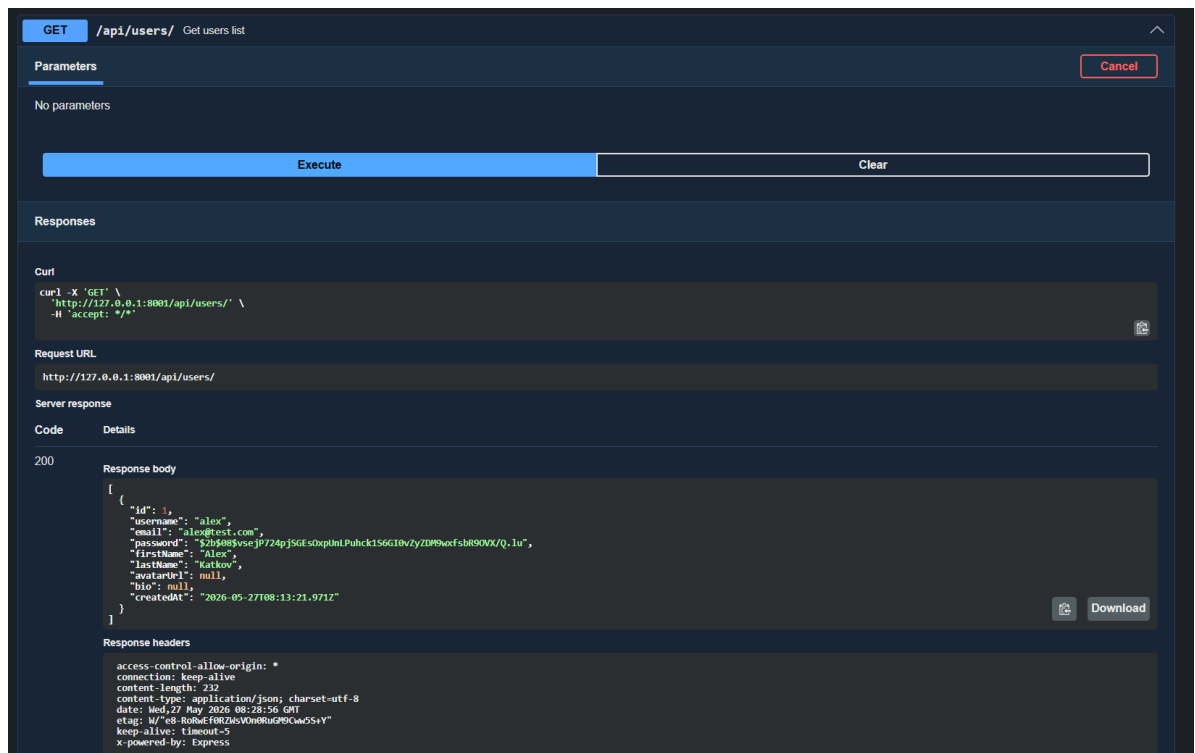


Рис. 8: Проверка получения ингредиента в Recipe Service

Для проверки Social Service был создан комментарий к рецепту. Перед созданием комментария был выполнен вход пользователя и получен JWT-токен. Затем токен был передан в заголовке Authorization.

Пример запроса на создание комментария:

```
Invoke-RestMethod '  
-Uri http://127.0.0.1:8000/api/comments/ '  
-Method POST '  
-ContentType "application/json" '  
-Headers @{ Authorization = "Bearer $token" } '  
-Body '{  
  "text":"Cool recipe",  
  "recipeId":1  
}',
```

После создания комментария endpoint получения комментариев вернул запись с текстом Cool recipe, userId = 1 и recipeId = 1.

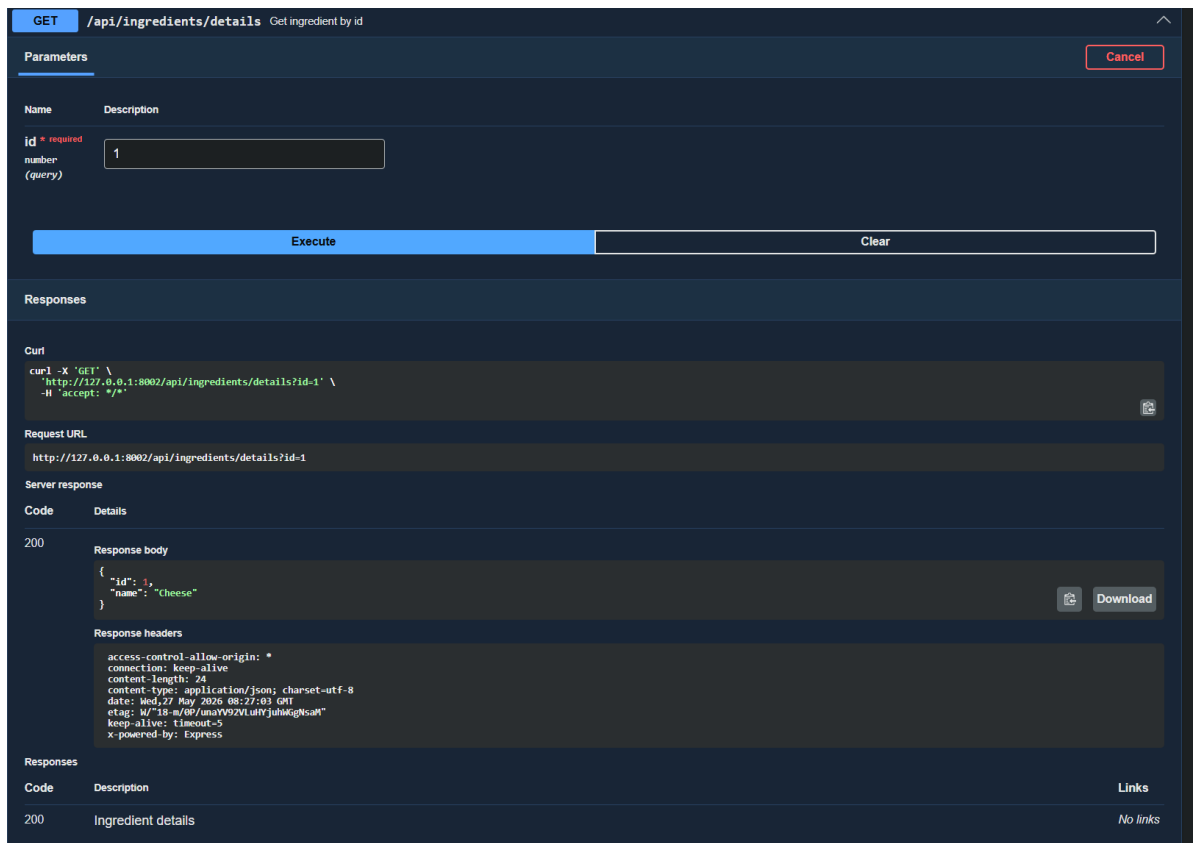


Рис. 9: Проверка получения комментариев в Social Service

8. Итоговая проверка

В результате тестирования было подтверждено, что:

- каждый микросервис запускается отдельно на своем порту;
- каждый микросервис использует собственную базу данных;
- API Gateway принимает запросы на порту 8000 и перенаправляет их в нужный сервис;
- Swagger UI доступен для каждого микросервиса;
- регистрация, авторизация, создание рецепта и создание комментария работают;
- JWT-токен используется для защищенных операций в Social Service.

Вывод

В ходе лабораторной работы было выполнено разделение монолитного backend-приложения на микросервисы. Были выделены Auth Service, Recipe Service и Social Service, а также реализован API Gateway как единая точка входа для клиентских запросов.

Для каждого микросервиса была создана отдельная база данных PostgreSQL что соответствует подходу database-per-service. Прямые связи между сущностями из разных сервисов были заменены на хранение идентификаторов, например authorId, userId и recipeId.

Работоспособность системы была проверена через Swagger UI и PowerShell-запросы. В результате были успешно выполнены регистрация пользователя, получение списка пользователей, создание ингредиента, создание рецепта и создание комментария с использованием JWT-токена.

Таким образом, цель лабораторной работы была достигнута: исходное backend-приложение было разделено на несколько независимых микросервисов с отдельными базами данных и единым API Gateway.